

Trabalho Final INF1022 2026.1 - Analizador Sintático ObsAct

Integrantes

- Arthur Barbosa - 2310394
- Gabriel Arruda - 2311723

1. Objetivo

O objetivo do trabalho foi desenvolver um analisador léxico e sintático para a linguagem ObsAct, capaz de receber um programa escrito em ObsAct e gerar um programa equivalente em outra linguagem. A linguagem alvo escolhida foi Python.

O analisador foi implementado com a biblioteca PLY (Python Lex-Yacc), que fornece ferramentas para construção de analisadores léxicos e sintáticos baseados em Lex/Yacc e permite a construção de um parser LALR(1).

2. Arquitetura da Implementação

O projeto foi dividido em cinco módulos principais:

- `lexer.py`: define os tokens da linguagem ObsAct e realiza a análise léxica.
- `parser.py`: define a gramática LALR(1), monta a AST e reconhece programas ObsAct.
- `semantic.py`: realiza validações semânticas, como declaração de dispositivos, observações, duplicatas, limites de tamanho e associações entre dispositivos e observações.
- `codegen.py`: gera o código Python equivalente a partir da AST validada.
- `obsact.py`: driver principal; lê o arquivo `.obsact`, executa `lexer`, `parser` e `semântica`, e escreve o arquivo `.py` gerado.

3. Funcionalidades Implementadas

- Declaração de dispositivos: `dispositivo: {namedevice}` e `dispositivo: {namedevice, observation}`.
- Atribuição via `set observation = VAR`.
- Atribuição via `set observation = ACTEXECUTE`, por exemplo `set estado_ventilador = verificar(ventilador)`.
- Atribuição com contexto de dispositivo: `set {namedevice, observation} = VAR`.
- Valores numéricos inteiros não negativos e valores booleanos TRUE/FALSE, incluindo variações de maiúsculas e minúsculas.
- Condicionais `se OBS então CMDs` e `se OBS então CMDs senão CMDs`.
- Blocos com múltiplos comandos dentro de condicionais e suporte a ifs aninhados.
- Condições compostas com `&&` e operadores `>`, `<`, `>=`, `<=`, `==` e `!=`.
- Ações ligar, desligar e verificar.
- Uso de `verificar` em condições, como `se verificar(umidificador) == 0 então ...`
- Envio de alerta simples e com variável.
- Envio de alerta sem parênteses, pois alguns exemplos do enunciado usam essa forma.
- Envio broadcast para múltiplos dispositivos.
- Inicialização automática de toda `observation` para zero.
- Geração das funções de runtime em Python: ligar, desligar, verificar e alerta.

4. Estado dos Dispositivos

O código Python gerado mantém um dicionário interno com o estado de cada dispositivo declarado. Ao executar `ligar namedevice`, o estado passa a ser 1. Ao executar `desligar namedevice`, o estado passa a ser 0. Ao executar `verificar(namedevice)`, o valor retornado corresponde ao estado atual do dispositivo.

Dessa forma, após executar ligar lampada, a chamada verificar(lampada) retorna 1; após executar desligar lampada, retorna 0.

5. Validações Semânticas

- Todo dispositivo usado deve ter sido declarado.
- Toda observação usada deve ter sido declarada ou criada por atribuição de retorno de ação.
- Dispositivos duplicados são rejeitados.
- namedevice deve conter somente letras e ter no máximo 100 caracteres.
- msg deve ter no máximo 100 caracteres.
- observation deve começar com letra e pode conter letras, números ou sublinhado.
- set {namedevice, observation} valida se a observation pertence ao dispositivo informado. A aceitação de sublinhado em nomes de observação foi incluída para cobrir exemplos do próprio enunciado, como estado_ventilador.

6. Gramática Final Utilizada

```
programa      -> devices cmds
devices       -> device devices | device
device        -> dispositivo device_open IDENT }
               | dispositivo device_open IDENT , IDENT }
device_open   -> : { | {
cmds          -> full_cmd cmds | full_cmd
full_cmd      -> obsact . | obsact | simple_cmd . | simple_cmd
simple_cmd     -> attrib | act
attrib        -> set IDENT = var
               | set IDENT = actexecute
               | set { IDENT , IDENT } = var
var -> NUMBER | TRUE | FALSE
obsact        -> se obs entao if_cmds
               | se obs então if_cmds senão if_cmds
if_cmds       -> simple_cmd . if_cmds | simple_cmd . | simple_cmd
               | obsact . if_cmds | obsact . | obsact if_cmds | obsact
obs -> IDENT oplogic var | IDENT oplogic var && obs |
      verificar ( IDENT ) oplogic var
      | verificar ( IDENT ) oplogic var && obs |
      verificar IDENT oplogic var
      | verificar IDENT oplogic var && obs
oplogic -> > | < | >= | <= | == | != act
-> actexecute
      | enviar alerta alert_args IDENT
      | enviar alerta alert_args para todos : namelist
actexecute -> ligar IDENT | desligar IDENT
      | verificar ( IDENT ) | verificar IDENT
alert_args -> ( STRING ) | ( STRING , IDENT ) | STRING | STRING , IDENT
namelist -> IDENT | IDENT , namelist
```

7. Alterações em Relação ao Enunciado

- A gramática foi fatorada e organizada para funcionar corretamente com PLY e LALR(1).
- O corpo do if foi implementado com o não-terminal if_cmds para permitir múltiplos comandos e ifs aninhados.
- verificar foi implementado principalmente na forma verificar(dev), usada nos exemplos, e também aceita verificar dev.

- Foram aceitos alertas sem parênteses porque alguns exemplos usam enviar alerta "msg" Monitor.
- Foi aceita declaração dispositivo {Monitor}, sem dois-pontos, porque aparece nos exemplos.
- Foram aceitos comandos simples sem ponto final, pois alguns exemplos finais do enunciado aparecem sem ponto.
- Foi aceito sublinhado em observações para nomes como estado_ventilador.

8. Testes Utilizados

Foram usados cinco arquivos .obsact na pasta testes/:

- exemplo1.obsact: declaração de dispositivos, set, condicional simples e ligar.
- exemplo2.obsact: verificar em atribuição, if com múltiplos comandos e if aninhado.
- exemplo3.obsact: if-else, alerta sem parênteses, ligar e desligar.
- exemplo4.obsact: set {dev, obs}, verificar em condição, if aninhado e broadcast.
- exemplo5.obsact: broadcast com variável para múltiplos dispositivos.

Também foram testados casos adicionais: comandos sem ponto final, verificar após ligar, rejeição de namedevice com mais de 100 caracteres e rejeição de set {dev, obs} quando a observação não pertence ao dispositivo.

9. Como Executar

```
pip install ply
python obsact.py entrada.obsact saida.py
python saida.py
```

Exemplo:

```
python obsact.py testes/exemplo1.obsact testes/exemplo1.py
python testes/exemplo1.py
```

Na nossa implementação, deixamos os casos de teste dentro de uma pasta “testes”. Todas as funcionalidades exigidas pelo enunciado foram implementadas e testadas: análise léxica, análise sintática LALR(1), geração de código Python, declaração de dispositivos, atribuições, ações, verificação de estado, alertas simples e com variável, broadcast, condicionais com múltiplos comandos, if-else, ifs aninhados e validações semânticas.